POLYNOMIÆ

REDUCTIONIS

PRINCIPIA

AUTOMATA

Syed Danial Haseeb

Contents

PROBLEM 1

MINESWEEPER

This problem is inspired by the single-player game *Minesweeper*, generalised to an arbitrary graph. Let G be an undirected graph, where each node either contains a single, hidden mine or is empty. The player chooses nodes, one by one. If the player chooses a node containing a mine, the player loses. If the player chooses an empty node, the player learns the number of neighbouring nodes containing mines. (A neighbouring node is one connected to the chosen node by an edge.) The player wins if and when all empty nodes have been so chosen.

In the *mine consistency problem*, you are given a graph G along with numbers labeling some of G's nodes. You must determine whether a placement of mines on the remaining nodes is possible, so that any node v that is labelled m has exactly m neighbouring nodes containing mines. Formulate this problem as a language and show that it is NP-complete.

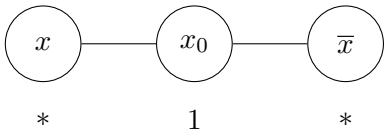
SOLUTION

$MCP = \{ \langle G, g \rangle \mid G$ is a graph, $G = (V, E)$, g is a function, $g : V \mapsto \mathbb{N} \cup \{*\}$, and a placement of mines on starred nodes is possible, so that all numbers are consistent. $\}$

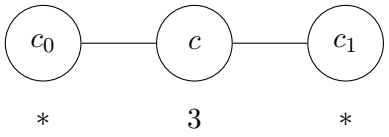
First, $MCP \in$ NP because we can guess an assignment of mines and verify that it is consistent, in polynomial time.

Second, we show that $3SAT \leq_{\mathrm{P}} MCP$. Given ϕ, convert it to a Minesweeper configuration as follows.

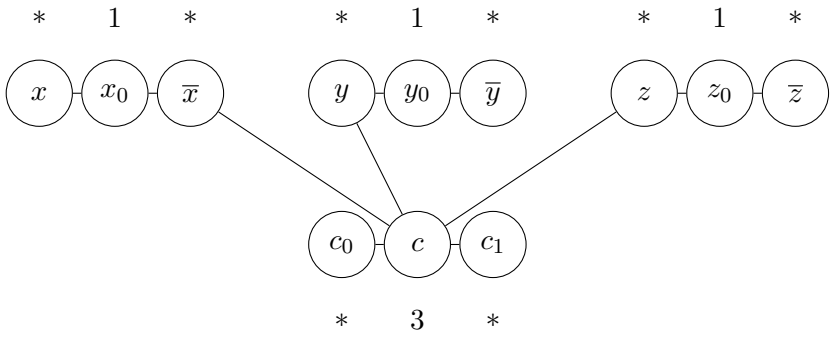
For each variable x, add nodes called x, x_0, and $\overline{x}$. Connect them with the edges (x, x_0) and $(x_0, \overline{x})$. Set $g(x) = g(\overline{x}) = *$ and $g(x_0) = 1$.



For each clause c, add nodes called c, c_0, and c_1. Connect them with the edges (c, c_0) and (c, c_1). Set $g(c_0) = g(c_1) = *$ and $g(c) = 3$.



Finally, connect node c to the nodes corresponding to the literals in clause c. For example, if $c = \overline{x} \vee y \vee z$, the corresponding gadget would look like this:



We show that ϕ is satisfiable if and only if there is a consistent placement of mines on starred nodes.

($\Longrightarrow$)

Take a satisfying assignment.

For each variable x assigned TRUE, put a mine on node x and no mine on node $\overline{x}$. For each variable x assigned FALSE, put a mine on node $\overline{x}$ and no mine on node x. This ensures that exactly one neighbour of x_0 has a mine.

For every clause c, the number of mine-carrying neighbours of node c is equal to the number of true literals in c. Since ϕ is satisfied, this number is 1, 2, or 3. Assign mines to c_0 and c_1, if necessary, to bring the number of mine-carrying neighbours of node c to 3.

($\Longleftarrow$)

Take a consistent mine placement.

For each variable x, exactly one of nodes x and $\overline{x}$ carries a mine because $g(x_0) = 1$. Assign variable x TRUE if node x carries a mine. Assign variable x FALSE if node $\overline{x}$ carries a mine.

Each clause c is satisfied because node c has three neighbours with mines. At least one of them is neither c_0 nor c_1. Hence, it has to be a node that corresponds to a literal in clause c. That literal is set to TRUE. Therefore, the clause is satisfied.

PROBLEM 2

SOLITARE

In the following solitaire game, you are given an $m \times m$ board. On each of its m^2 positions lies either a blue stone, a red stone, or nothing at all. You play by removing stones from the board until each column contains only stones of a single colour and each row contains at least one stone. You win if you achieve this objective. Winning may or may not be possible, depending upon the initial configuration. Let

$$\textit{SOLITAIRE} = \{\, \langle G \rangle \mid G \text{ is a winnable game configuration} \,\}.$$

Prove that *SOLITAIRE* is NP-complete.

SOLUTION

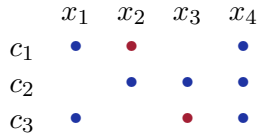
First, *SOLITAIRE* $\in$ NP because we can verify that a solution works, in polynomial time.

Second, we show that *3SAT* $\leq_{\mathrm{P}}$ *SOLITAIRE*. Given ϕ with n variables, $x_1, x_2, \ldots, x_n$, and m clauses, $c_1, c_2, \ldots, c_m$, construct the following $m \times n$ game, G. We assume that ϕ has no clauses that contain both x_i and $\overline{x_i}$ because such clauses may be removed without affecting satisfiability.

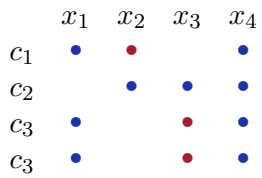
If variable x_i is in clause c_j, put a blue stone in row c_j, column x_i. If variable $\overline{x_i}$ is in clause c_j, put a red stone in row c_j, column x_i. For example, if

$$\phi = \underbrace{(x_1 \vee \overline{x_2} \vee x_4)}_{c_1} \wedge \underbrace{(x_2 \vee x_3 \vee x_4)}_{c_2} \wedge \underbrace{(x_1 \vee \overline{x_3} \vee x_4)}_{c_3},$$

then the corresponding game configuration would look like this:



We can make the board square by repeating a row or adding a blank column, as necessary, without affecting solvability:



We show that ϕ is satisfiable if and only if G has a solution.

($\Longrightarrow$)

Take a satisfying assignment.

If variable x_i is TRUE, remove the red stones from the corresponding column x_i. If variable x_i is FALSE, remove the blue stones from the corresponding column x_i.

Thus, only the stones corresponding to true literals remain. Since every clause has a true literal, every row has a stone.

($\Longleftarrow$)

Take a game solution.

If the red stones were removed from column x_i, set the corresponding variable x_i TRUE. If the blue stones were removed from column x_i, set the corresponding variable x_i FALSE.

Every row has a stone remaining, so every clause has a true literal. Therefore, ϕ is satisfied.

PROBLEM 3

SET-SPLITTING

Let *SET-SPLITTING* $= \{ \langle S, C \rangle \mid S$ is a finite set and $C = \{ C_1, C_2, \ldots, C_k \}$ is a collection of subsets of S, for some $k > 0$, such that elements of S can be coloured *red* or *blue* so that no C_i has all its elements coloured with the same colour $\}$. Show that *SET-SPLITTING* is NP-complete.

SOLUTION

First, *SET-SPLITTING* $\in$ NP because a colouring of the elements can be verified to have the desired properties in polynomial time.

Second, we show that *3SAT* $\leq_{\mathrm{P}}$ *SET-SPLITTING*. Given ϕ with n variables, $x_1, x_2, \ldots, x_n$, and m clauses, $c_1, c_2, \ldots, c_m$, we set

$$S = \{ x_1, \overline{x_1}, x_2, \overline{x_2}, \ldots, x_n, \overline{x_n}, y \}.$$

In other words, S contains an element for each literal (two for each variable) and a special element, y.

For every clause, let C_i be a subset of S containing the elements corresponding to the literals in c_i and the special element, $y \in S$. For example,

$$c_3 = x_1 \vee \overline{x_2} \vee x_3 \implies C_3 = \{ x_1, \overline{x_2}, x_3, y \}.$$

We also add the subsets $C_{x_i} = \{ x_i, \overline{x_i} \}$ for each literal. Finally,

$$C = \bigcup_{i=1}^{m} C_i \cup \bigcup_{i=1}^{n} C_{x_i} = \{ C_1, C_2, \ldots, C_m, C_{x_1}, C_{x_2}, \ldots, C_{x_n} \}.$$

We show that ϕ is satisfiable if and only if $\langle S, C \rangle \in$ *SET-SPLITTING*.

($\implies$)

Take a satisfying assignment.

If variable x_i is TRUE, colour the corresponding element $x_i \in S$ *blue*. If variable x_i is FALSE, colour the corresponding element $x_i \in S$ *red*. Colour y *red*.

Thus, every subset of S has at least one blue element (since every clause has at least one true literal) and it also contains one red element, y.

In addition, every subset C_{x_i} has exactly one red element and one blue. Therefore, $\langle S, C \rangle \in$ *SET-SPLITTING*.

($\Longleftarrow$)

Let $\langle S, C \rangle \in$ *SET-SPLITTING*.

Look at the colouring for S. Set each literal to TRUE that is coloured differently from y. Set each literal to FALSE that is coloured the same as y. Thus, ϕ is satisfied.

PROBLEM 4

SCHEDULING

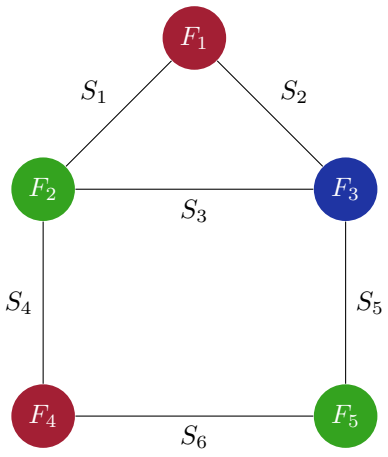
Consider the following scheduling problem. You are given a list of final exams, $F_1, F_2, \ldots, F_k$, to be scheduled, and a list of students, $S_1, S_2, \ldots, S_\ell$. Each student is taking some specified subset of these exams. You must schedule these exams into slots so that no student is required to take two exams in the same slot. The problem is to determine if such a schedule exists that uses only h slots. Formulate this problem as a language and show that this language is NP-complete.

SOLUTION

SCHEDULE $= \{ \langle F, S, h \rangle \mid F$ is a list of finals, S is a list of finals subsets of finals each student is taking, and finals can be scheduled into h slots so that no student is taking two exams in the same slot $\}$.

First, *SCHEDULE* $\in$ NP because we can verify that no student is taking two exams in the same slot, in polynomial time.

Second, we show that *3COLOUR* $\leq_{\mathrm{P}}$ *SCHEDULE*. Given a graph, $G = (V, E)$, convert it to $\langle F, S, h \rangle$ where $F = V$, $S = E$, and $h = 3$. For example:



We show that G is 3-colourable if and only if $\langle V, E, 3 \rangle \in$ *SCHEDULE*.

($\Longrightarrow$)

Take a 3-colourable graph.

We can get a schedule for $\langle V, E, 3\rangle$ by assigning finals that correspond to nodes coloured red to slot 1, blue to slot 2, and green to slot 3. Adjacent nodes are coloured by different colours, so corresponding exams taken by the same student are assigned to different slots. Therefore, $\langle V, E, 3\rangle \in \mathit{SCHEDULE}$.

($\Longleftarrow$)

Take a valid schedule for $\langle V, E, 3\rangle$.

We can get a 3-colouring by assigning colours according to the slots of corresponding exams. Since exams taken by the same student are scheduled in different slots, adjacent nodes are assigned different colours.

PROBLEM 5

DOMINATING-SET

A subset of the nodes of a graph G is a *dominating set* if every other node of G is adjacent to some node in the subset. Let

$$\textit{DOMINATING-SET} = \{ \langle G, k \rangle \mid G \text{ has a dominating set with } k \text{ nodes} \}.$$

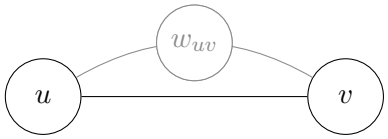
Show that it is NP-complete by giving a reduction from *VERTEX-COVER*.

SOLUTION

First, we need to show *DOMINATING-SET* $\in$ NP. The certificate is simply the dominating set.

Second, we give a polynomial time mapping reduction from *VERTEX-COVER* to *DOMINATING-SET*. The input to the reduction is a pair $\langle G, k \rangle$ and the reduction produces the pair $\langle G', k' \rangle$ as output where the graph G' and integer k' are as follows.

For each edge (u, v) in G, add a node w_{uv} in G' and connect it to u and v:



Let α be the number of *isolated* nodes in G. Set

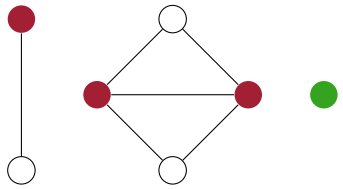
$$k' = k + \alpha.$$

Observe that G has a vertex cover with k nodes if and only if G' has a dominating set with k' nodes.

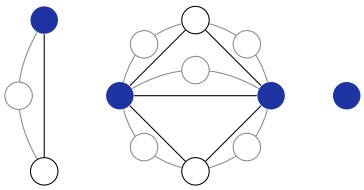
Let V_{C} be the set of k nodes that is a vertex cover for G, V_{I} be the set of α isolated nodes in G, and V_{D} be the set of k' nodes that is a dominating set for G'. Then

$$V_{\text{D}} = V_{\text{C}} \cup V_{\text{I}}.$$

For example, let G be the following graph that has a vertex cover with $k = 3$ nodes and $\alpha = 1$ isolated node:



Then G' would be the following graph that has a dominating set with $k' = k + \alpha = 4$ nodes:



All isolated nodes are in V_D.

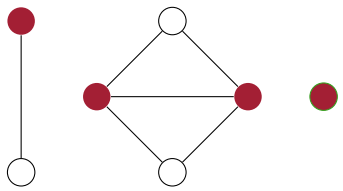
Each of the special nodes w_{uv} in G' correspond to an edge (u, v) in G; so either u or v (or both of them) is in V_C. Since $V_C \subseteq V_D$, every special node is adjacent to a node in V_D.

The remaining nodes are the nonisolated nodes from the original graph G. Each of these are nonisolated, so they have at least one edge in G. Therefore, either they are in V_C or all of their neighbours are. Either way, they are in V_D or adjacent to a node in it.

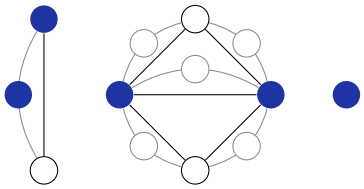
Note that the dominating set may have fewer nodes than k':

$$\begin{aligned} |V_D| &= |V_C \cup V_I| \\ &\leq |V_C| + |V_I| \\ &\leq k + \alpha \\ &\leq k' \end{aligned}$$

This will happen if the vertex cover includes isolated vertices. For example, consider the the graph G again, but with a vertex cover with $k = 4$ nodes, including the $\alpha = 1$ isolated node:

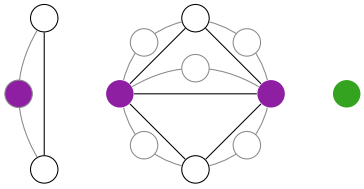


G', however, would still have a dominating set with 4 nodes, instead of $k' = k + \alpha = 5$. In such cases, simply add any of the remaining nodes to the dominating set such that $|V_D| = k'$:

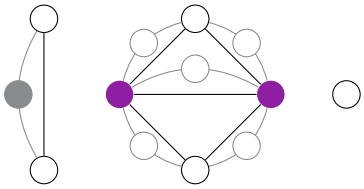


Thus, $\langle G', k' \rangle \in$ *DOMINATING-SET* if $\langle G, k \rangle \in$ *VERTEX-COVER*.

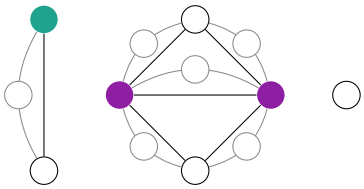
If G' has a dominating set with k' nodes, it would consist of α isolated nodes and k nonisolated nodes. For example, here $\alpha = 1$ and $k = 3$:



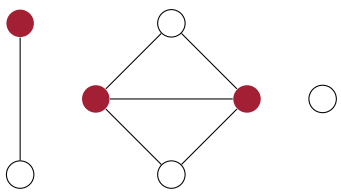
Of these k nonisolated nodes, some may be special nodes that don't exist in G:



For each special node w_{uv} in this set of k nodes, replace it by u (or v):



By making this replacement, we still have a dominating set for G' because u is adjacent to the same nodes as w_{uv}, (and potentially more). After this replacement, we get the vertex cover for G with k nodes:



Thus, $\langle G, k \rangle \in$ *VERTEX-COVER* if $\langle G', k' \rangle \in$ *DOMINATING-SET*.

PROBLEM 6

STATES & STRINGS

Show that the following problem is NP-complete. You are given a set of states $Q = \{ q_0, q_1, \ldots, q_\ell \}$ and a collection of pairs $\{ (s_1, r_1), (s_2, r_2), \ldots, (s_k, r_k) \}$ where the s_i are distinct strings over $\Sigma = \{ \mathtt{0}, \mathtt{1} \}$, and the r_i are (not necessarily distinct) members of Q. Determine whether a DFA $M = (Q, \Sigma, \delta, q_0, F)$ exists where $\hat{\delta}(q_0, s_i) = r_i$ for each i. Here, $\hat{\delta}(q, s)$ is the state that M enters after reading s, starting at state q. (Note that F is irrelevant here.)

SOLUTION

Let K be the language describing the set of inputs for which a DFA exists with the specified properties.

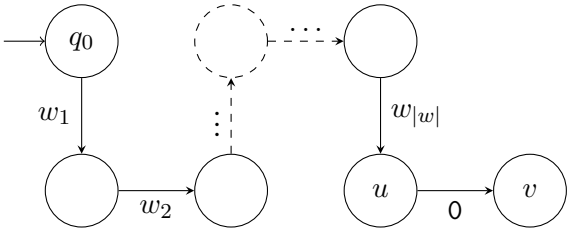
Clearly, $K \in$ NP.

We give a polynomial time mapping reduction from *3SAT* to K. The input to the reduction is $\langle \phi \rangle$ with n variables and the reduction produces the pair $\langle P, Q \rangle$ as output where $P = \{ (s_1, r_1), (s_2, r_2), \ldots, (s_k, r_k) \}$ is a collection of pairs designed to restrict the DFA and Q is a collection of states as follows.

We give convenient names to the states added in Q:

q_0	starting state
q_1	first variable in ϕ
q_2	second variable in ϕ
$\vdots$	$\vdots$
q_n	nth variable in ϕ
h	helper state
x_0	variable assigned FALSE
x_1	variable assigned TRUE
ℓ_0	literal assigned FALSE
ℓ_1	literal assigned TRUE
a	assignment-enforcing state
s	satisfiability state

Observe that we can enforce a 0-transition from state u to v by adding the pairs (w, u) and $(w0, v)$ in P, where w is some string.



Here are the pairs we add in P, along with the transitions they enforce:

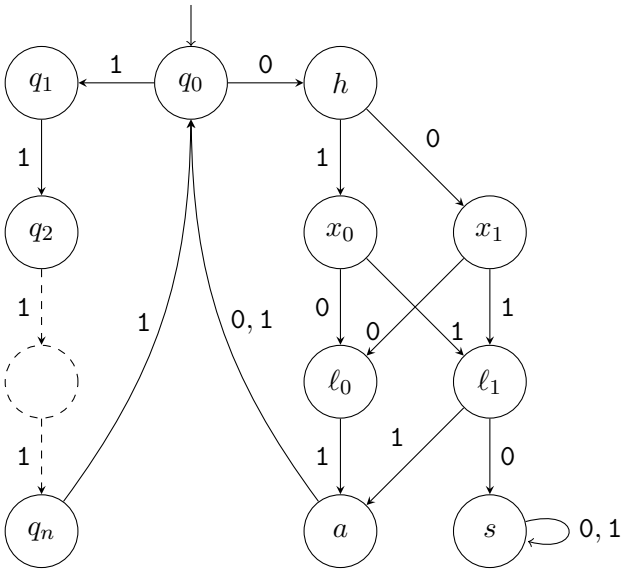
$$
\begin{array}{rl}
(1, q_1) & \delta(q_0, 1) = q_1 \\
(11, q_2) & \delta(q_1, 1) = q_2 \\
(1^3, q_3) & \delta(q_2, 1) = q_3 \\
(1^4, q_4) & \delta(q_3, 1) = q_4 \\
\vdots & \vdots \\
(1^n, q_n) & \delta(q_{n-1}, 1) = q_n \\
(1^{n+1}, q_0) & \delta(q_n, 1) = q_0 \\
(0, h) & \delta(q_0, 1) = q_1 \\
(00, x_1) & \delta(h, 0) = x_1 \\
(01, x_0) & \delta(h, 1) = x_0 \\
(000, \ell_1) & \delta(x_1, 0) = \ell_1 \\
(001, \ell_0) & \delta(x_1, 1) = \ell_0 \\
(010, \ell_1) & \delta(x_0, 0) = \ell_0 \\
(011, \ell_0) & \delta(x_0, 1) = \ell_1 \\
(0000, s) & \delta(\ell_1, 0) = s \\
(0001, a) & \delta(\ell_1, 1) = a \\
(0010, q_0) & \delta(\ell_0, 0) = q_0 \\
(0011, a) & \delta(\ell_0, 1) = a \\
(00000, s) & \delta(s, 0) = s \\
(00001, s) & \delta(s, 1) = s \\
(00010, q_0) & \delta(a, 0) = q_0 \\
(00011, q_0) & \delta(a, 1) = q_0
\end{array}
$$

The 0-transitions from $q_1, q_2, \ldots, q_n$ encode the assignment. Transitioning to x_1 indicates TRUE, and x_0 indicates FALSE. To force one transition or the other, we add the pairs $(1011, a), (11011, a), (1^3011, a), (1^4011, a), \ldots, (1^n011, a)$. This works because x_0 and x_1 are the only two states that have a path to a by reading 11.

Additionally, consider a clause $l_1 \vee l_2 \vee l_3$ in ϕ. Each l_i is a literal x_j or $\overline{x_j}$. We force this clause to be satisfied by adding the pair $(t_1t_2t_3, s)$ where each $t_i = 1^j 0 b 0$ and

$$b = \begin{cases} 0, & l_i = x_j, \\ 1, & l_i = \overline{x_j}. \end{cases}$$

With these collections of pairs P and states Q, we have essentially constructed the following (partial) DFA:



The missing transitions in this automaton are the 0-transitions from q_1 to q_n, that correspond exactly to the assignment of variables in ϕ. We know these transitions would have to end up on either x_0 or x_1.

This would only be possible if ϕ were satisfiable since we have also enforced that each clause is true by placing the last restriction in P.

PROBLEM 7

SIMULATING NTMS

Let

$$U = \{ \langle M, x, \#^t \rangle \mid \text{NTM } M \text{ accepts } x \text{ within } t \text{ steps on at least one branch} \}.$$

Note that M isn't required to halt on all branches. Show that U is NP-complete.

SOLUTION

First, $U \in$ NP because on input $\langle M, x, 1^t \rangle$, a nondeterministic algorithm can simulate M on x (making nondeterministic branches when M does) for t steps and ACCEPT if M accepts. Doing so takes polynomial time in the size of the input because t appears in unary.

Second, we show that *3SAT* $\leq_{\text{P}} U$. Let NTM M decide *3SAT* in cn^k time for some constants c and k. Given ϕ with n variables, convert it to $\left\langle M, \phi, 1^{cn^k} \right\rangle$.

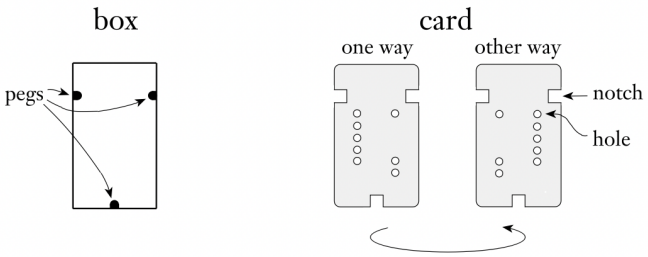
By the definition of M,

$$\left\langle M, \phi, 1^{cn^k} \right\rangle \in U \iff \langle \phi \rangle \in \textit{3SAT}.$$

PROBLEM 8

PUZZLE

You are given a box and a collection of cards as indicated in the following figure.



Because of the pegs in the box and the notches in the cards, each card will fit in the box in either of two ways. Each card contains two columns of holes, some of which may not be punched out. The puzzle is solved by placing all the cards in the box so as to completely cover the bottom of the box (i.e., every hole position is blocked by at least one card that has no hole there). Let

$$\textit{PUZZLE} = \{\, \langle c_1, c_2, \ldots, c_k \rangle \mid c_i \text{ is a card, this set of cards has a solution} \,\}.$$

Show that *PUZZLE* is NP-complete.

SOLUTION

First, $\textit{PUZZLE} \in$ NP because we can verify a solution in polynomial time.

Second, we show that $\textit{3SAT} \leq_{\mathrm{P}} \textit{PUZZLE}$. Given ϕ with n variables and m clauses, convert it to a set of cards as follows.

Let $x_1, x_2, \ldots, x_n$ be the variables in ϕ $c_1, c_2, \ldots, c_m$ be the clauses. Say $\ell \in c_j$ if ℓ is one of the literals in c_j.

Let each card have columns 1 and 2 with m possible holes in each column. When placing a card in the box, say that a card is *face up* if column 1 is on the left; otherwise *face down.*

Create one card for each x_i. In column 1, punch out all holes except any hole j such that $x_i \in c_j$. In column 2, punch out all holes except any hole j' such that $\overline{x_i} \in c_{j'}$.

Create an *extra* card with all holes in column 1 punched out and no hole in column 2 punched out.

We show that ϕ is satisfiable if and only if this set of cards has a solution.

($\Longrightarrow$)

Take a satisfying assignment.

For each x_i assigned TRUE, put its corresponding card face up. For each x_i assigned FALSE, put its card face down.

Put the extra card face up.

Every hole in column 1 is covered because the associated clause has a literal assigned TRUE. Every hole in column 2 is covered by the extra card.

($\Longleftarrow$)

Take a puzzle solution.

Flip the deck so that the extra card covers column 2.

Assign the variable TRUE if the corresponding card is face up or assign it FALSE if face down.

Since every hole in column 1 is covered, every clause contains at least one literal assigned TRUE.

PROBLEM 9

LONG PATHS

Let G represent an undirected graph. Also let

$$\mathit{LPATH} = \{\, \langle G, a, b, k\rangle \mid G \text{ has a simple path of length at least } k \text{ from } a \text{ to } b \,\}.$$

Show that *LPATH* is NP-complete.

SOLUTION

First, $\mathit{LPATH} \in$ NP because we can verify a simple path of length at least k from a to b in polynomial time.

Second, we show that $\mathit{UHAMPATH} \leq_{\mathrm{P}} \mathit{LPATH}$. Given $\langle G, a, b\rangle$, convert it to $\langle G, a, b, k\rangle$, where k is the number of nodes in G.

($\Longrightarrow$)

Take an undirected Hamiltonian path from a to b.

Since G has k nodes, it has a Hamiltonian path from a to b of length k. Thus, $\langle G, a, b, k\rangle \in \mathit{LPATH}$.

($\Longleftarrow$)

Take a simple path that has length at least k.

Since G has only k nodes, this path is Hamiltonian. Thus $\langle G, a, b\rangle \in \mathit{UHAMPATH}$.

PROBLEM 10

DOUBLE-SAT

Let *DOUBLE-SAT* $= \{ \langle \phi \rangle \mid \phi$ has two satisfying assignments $\}$. Show that *DOUBLE-SAT* is NP-complete.

SOLUTION

First, we show that *DOUBLE-SAT* $\in$ NP. The certificate is simply the pair of assignments.

Second, we show that *SAT* $\leq_{\mathrm{P}}$ *DOUBLE-SAT*. Given ϕ, convert it to

$$\phi' = \phi \wedge (x \vee \overline{x}),$$

where x is a new variable.

($\Longrightarrow$)

Take a satisfying assignment for ϕ.

ϕ' has at least two satisfying assignments that can be obtained from the original satisfying assignment of ϕ by changing the value of x.

($\Longleftarrow$)

Take a satisfying assignment for ϕ'.

ϕ must also be satisfiable since x does not appear in it.

PROBLEM 11

HALF-CLIQUE

Let *HALF-CLIQUE* $= \{ \langle G \rangle \mid G$ is an undirected graph having a complete subgraph with at least $\frac{m}{2}$ nodes, where m is the number of nodes in $G \}$. Show that *HALF-CLIQUE* is NP-complete.

SOLUTION

We give a polynomial time mapping reduction from *CLIQUE* to *HALF-CLIQUE*. The input to the reduction is a pair $\langle G, k \rangle$ and the reduction produces the graph $\langle H \rangle$ as output where H is as follows.

If G has m nodes and $k = \frac{m}{2}$, then $H = G$.

If $k < \frac{m}{2}$, then H is the graph obtained from G by adding j nodes, each connected to every one of the original nodes and to each other, where $j = m - 2k$. Thus, H has $m + j = 2m - 2k$ nodes.

Observe that G has a k-clique if and only if H has a clique of size $k + j = m - k$.

If $k > \frac{m}{2}$, then H is the graph obtained by adding j nodes to G without any additional edges, where $j = 2k - m$. Thus, H has $m + j = 2k$ nodes.

We also need to show *HALF-CLIQUE* $\in$ NP. The certificate is simply the clique.

PROBLEM 12

CONJUNCTIVE NORMAL FORM

Let $CNF_k = \{ \langle \phi \rangle \mid \phi$ is a satisfiable cnf-formula where each variable appears in at most k places $\}$. Show that CNF_3 is NP-complete.

SOLUTION

First, we show that $CNF_3 \in$ NP. The certificate is simply the assignment.

Second, we show that $3SAT \leq_{\mathrm{P}} CNF_3$. Given ϕ, make the following changes.

For each variable x that appears $k > 3$ times in ϕ, replace it with the k variables $x_1, x_2, \ldots x_k$ and add the clauses

$$(\overline{x_1} \vee x_2) \wedge (\overline{x_2} \vee x_3) \wedge \cdots \wedge (\overline{x_k} \vee x_1).$$

These are equivalent to

$$(x_1 \implies x_2) \wedge (x_2 \implies x_3) \wedge \cdots \wedge (x_k \implies x_1).$$

This restricts the new variables to the same truth value while not affecting satisfiability.

Moreover, in the new formula, x does not appear at all, while $x_1, x_2, \ldots x_k$ appear exactly three times (twice in the *cycle* above and once as a replacement for x).

PROBLEM 13

≠SAT

Let ϕ be a 3cnf-formula. An $\neq$-assignment to the variables of ϕ is one where each clause contains two literals with unequal truth values. In other words, an $\neq$-assignment satisfies ϕ without assigning three true literals in any clause.

Let $\neq SAT$ be the collection of 3cnf-formualæthat have an $\neq$-assignment. Show that we obtain a polynomial time reduction from *3SAT* to $\neq SAT$ by replacing each clause

$$c_i = y_1 \vee y_2 \vee y_3$$

with the two clauses

$$(y_1 \vee y_2 \vee z_i) \wedge (\overline{z_i} \vee y_3 \vee b),$$

where z_i is a new variable for each clause c_i and b is a single new additional variable.

SOLUTION

To prove that the given reduction works, we need to show that if ϕ is mapped to ϕ', then ϕ is satisfiable if and only if ϕ' has a $\neq$-assignment.

If ϕ is satisfiable, we can obtain an $\neq$-assignment for ϕ' by extending the assignment to ϕ so that we assign z_i TRUE if both literals y_1 and y_2 in clause c_i of ϕ are assigned FALSE. Otherwise we assign z_i to FALSE. Finally, we assign b FALSE.

If ϕ' has an $\neq$-assignment, each clause has at least one literal assigned TRUE and at least one assigned FALSE. The negation of this assignment preserves this property, so it too is an $\neq$-assignment.

Therefore, this assignment cannot assign all of y_1, y_2, and y_3 FALSE, because doing so would force one of the clauses in ϕ' to have all literals FALSE. Hence, restricting this assignment to the variables of ϕ gives a satisfying assignment.

PROBLEM 14

MAX-CUT

A *cut* in an undirected graph is a separation of the vertices V into two disjoint subsets S and T. The size of the cut is the number of edges that have one endpoint in S and the other in T. Let

$$\textit{MAX-CUT} = \{ \langle G, k \rangle \mid G \text{ has a cut of size } k \text{ or more} \}.$$

Show that *MAX-CUT* is NP-complete.

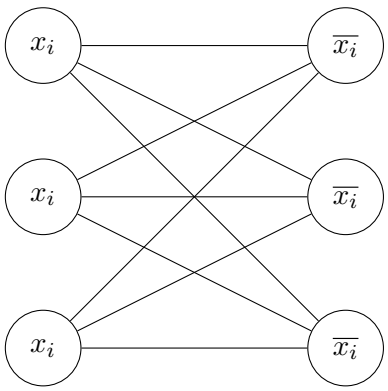
(Hint: show that $\neq$*SAT* $\leq_{\mathrm{P}}$ *MAX-CUT*. The variable gadget for variable x is a collection of $3m$ nodes labelled with x and another $3m$ nodes labelled $\overline{x}$, where m is the number of clauses. All nodes labelled x are connected with all nodes labelled x. The clause gadget is a triangle of three edges connecting three nodes labelled with the literals appearing in the clause. Do not use the same node in more than one clause gadget. Prove that this reduction works.)

SOLUTION

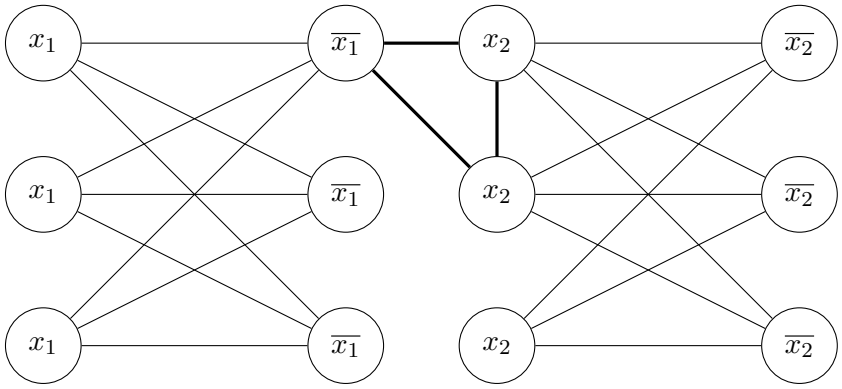
First, *MAX-CUT* $\in$ NP because we can verify the cut and check that it has size at least k, in polynomial time.

Second, we show that $\neq$*SAT* $\leq_{\mathrm{P}}$ *MAX-CUT* by using the reduction in the hint. Given ϕ with n variables and m clauses, we build a graph G with $6mn$ nodes.

Each variable x_i corresponds to $6c$ nodes; $3c$ labelled x_i and $3c$ labelled $\overline{x_i}$. Connect each node x_i with each node $\overline{x_i}$ for a total of $9c^2$ edges.



For each clause, select three nodes labelled by the literals in that clause and connect them. Avoid selecting the same node more than once (which is always possible since each node has three copies). For example, the clause $\overline{x_1} \vee x_2 \vee x_2$ would have the following **gadget**:



Set

$$k = 9c^2v + 2c$$

We show that ϕ has an ne-assignment if and only if G has a cut of size at least k.

($\Longrightarrow$)

Take an $\neq$-assignment for ϕ.

Place all nodes corresponding to variables assigned TRUE on one side and the rest on the other side. Then the cut contains all $9c^2v$ edges between literals and their negations.

There are two edges for each clause because because it contains at least one literal assigned TRUE and one FALSE, yielding a total of $9c^2v + 2c = k$ edges.

($\Longleftarrow$)

Take a cut of size k.

Observe that a cut can contain at most two edges for each clause, because at least two of the nodes corresponding to its literals must be on the same side. Thus, the clause edges can contribute at most $2c$ edges to the cut.

That leaves us with $9c^2v$ other edges. Since the cut is of size $k = 9c^2v + 2c$, all these other edges must be in the cut and all the clauses must contribute their maximum amount of edges.

Thus, all nodes labelled by the same literal appear on the same side. By selecting either side of the cut and assigning its corresponding literals TRUE, we obtain an assignment.

Since each clause contributes two edges to the cut, it must have nodes appearing on both sides. Therefore, it must have at least one literal assigned TRUE and one FALSE. Thus, ϕ has an $\neq$-assignment.